

Dataintegration och API-kontrakt

Flask + SQLite + mock-sensor. Steg för steg, en fil i taget.

Så här funkar flödet: mock-sensorn postar temperaturer till Flask-API:et, API:et validerar mot kontraktet i `openapi.yaml` och sparar i SQLite. Datan överlever omstart eftersom den ligger på disk och inte i minnet.

Du behöver **tre terminalfönster** när allt är på plats: ett för API:et, ett för sensorn, ett för dina kommandon.

STEG 1

Mapp och paket

```
mkdir -p iot-integration
cd iot-integration
pip install flask requests
```

VERIFIERA

```
python -c "import flask, requests; print('ok')"
```

STEG 2

Svara på frågorna i Del 1

Ingen kod här, bara läsning av kontraktet.

FRÅGA	SVAR
Vilka endpoints finns?	<code>POST /sensor-data</code> och <code>GET /temperature</code>
Vilka fält krävs vid POST?	<code>device</code> (sträng) och <code>temp</code> (tal)
Vilket format har temperaturen?	Decimaltal i Celsius, t.ex. <code>22.5</code> – inte strängen <code>"22.5"</code>

Poängen med kontraktet: sensorn och API:et kan skrivas av olika personer, i olika språk, men båda utgår från samma fil.

STEG 3

Skapa database.py

Labbandledningen klipps av mitt i `save_reading()`. Här är hela filen:

```
import sqlite3

def get_conn():
    conn = sqlite3.connect('sensor_data.db')
    conn.row_factory = sqlite3.Row # gör att raderna kan bli dictionaries
    return conn

def init_db():
    conn = get_conn()
    conn.execute('''
        CREATE TABLE IF NOT EXISTS readings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            device TEXT NOT NULL,
            temp REAL NOT NULL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    conn.commit()
    conn.close()

def save_reading(device, temp):
    conn = get_conn()
    conn.execute('INSERT INTO readings (device, temp) VALUES (?, ?)', (device, temp))
    conn.commit()
    conn.close()

def get_readings():
    conn = get_conn()
    rows = conn.execute('SELECT * FROM readings ORDER BY id DESC LIMIT 200').fetchall()
    conn.close()
    return [dict(r) for r in rows]
```

Använd alltid `?`-parametrar i SQL-frågorna, aldrig f-strings – annars öppnar du för SQL-injektion.

VERIFIERA

```
python -c "import database; database.init_db(); database.save_reading('test',21.5); print(database.get_readings())"
```

STEG 4

Skapa app.py

Den här filen visas inte alls i labbhandledningen.

```
from flask import Flask, request, jsonify
import database

app = Flask(__name__)

@app.route('/sensor-data', methods=['POST'])
def sensor_data():
    data = request.get_json()

    # Validera mot kontraktet
    if 'device' not in data or 'temp' not in data:
        return jsonify({'status': 'error', 'message': 'device och temp krävs'}), 400
    if not isinstance(data['temp'], (int, float)):
        return jsonify({'status': 'error', 'message': 'temp måste vara ett tal'}), 400

    database.save_reading(data['device'], data['temp'])
    return jsonify({'status': 'ok'})

@app.route('/temperature')
def temperature():
    return jsonify(database.get_readings())

if __name__ == '__main__':
    database.init_db()
    app.run(debug=True, port=5000)
```

Valideringen är hela poängen. Utan den kastar `data['device']` ett `KeyError` och du får `500 Internal Server Error`. Det är fel svar – felet ligger hos klienten som skickade trasig data, alltså ska det vara `400`.

STEG 5

Starta API:et

```
python app.py
```

VERIFIERA
Öppna `http://localhost:5000/temperature` i webbläsaren. Du ska se `[]` eller din testrad från steg 3.

STEG 6

Skapa mock_sensor.py

Också avklippt i handledningen – sista raden ska vara ett vanligt `print`.

```
import requests, time, random

API_URL = "http://localhost:5000/sensor-data"
DEVICE_ID = "sensor-mock-01"

def run_sensor():
    print(f"Startar mock-sensor: {DEVICE_ID}")
    while True:
        payload = {
            "device": DEVICE_ID,
            "temp": round(random.uniform(18.0, 26.0), 2)
        }
        try:
            response = requests.post(API_URL, json=payload)
            if response.status_code == 200:
                print(f"Skickat: {payload['temp']}°C - Status: {response.json()['status']}")
            else:
                print(f"Fel vid sändning: {response.status_code}")
        except Exception as e:
            print(f"Kunde inte kontakta API:et: {e}")
        time.sleep(2)

if __name__ == '__main__':
    run_sensor()
```

STEG 7

Kör båda samtidigt

Nytt terminalfönster, med API:et fortfarande igång:

```
python mock_sensor.py
```

VERIFIERA

I sensorfönstret ska det rulla: Skickat: 22.4°C - Status: ok . Låt gå en minut.

STEG 8

Persistenstestet

- Ctrl+C i API-fönstret
- python app.py igen
- Ladda om http://localhost:5000/temperature

Datan ligger kvar. Den finns i filen sensor_data.db på disk, inte i en Python-lista i minnet. Det är hela skillnaden mot förra lektionens simulering.

STEG 9

Fördjupningsuppgifterna

1. Schema-validering

Ändra i sensorn så att kontraktet bryts, t.ex. "temp": str(round(...)) eller ta bort raden med device . API:et ska svara 400 och inte spara något.

2. Flera sensorer

Kopiera sensorn till mock_sensor2.py , byt DEVICE_ID till sensor-mock-02 och kör båda samtidigt. Databasen behöver inte ändras – device är bara en kolumn, alla rader hamnar i samma tabell.

3. Tidsfiltrering

Byt SQL-frågan i get_readings() :

```
SELECT * FROM readings
WHERE timestamp >= datetime('now', '-1 hours')
ORDER BY id DESC
```

Tidszonsfällan. SQLite skriver CURRENT_TIMESTAMP i UTC, inte i svensk lokaltid. Därför måste du jämföra mot datetime('now') som också är UTC. Blandar du UTC och lokaltid får du tomma resultat, eller data som ser ut att komma från framtiden – ett klassiskt fel i loggningssystem.

4. Visualisering

En HTML-sida som hämtar /temperature med fetch() och ritar upp datan i en tabell eller med Chart.js. Lägg den i en static/ -mapp och servera den från Flask.

Filstruktur när du är klar

```
iot-integration/
├─ app.py           # API:et
├─ database.py      # SQLite-hjälpfil
├─ mock_sensor.py   # sensor-simulator
├─ openapi.yaml     # kontraktet
├─ static/index.html # vy för historisk data (fördjupning)
└─ sensor_data.db   # skapas automatiskt
```

Sammanfattning att lämna in

- Kontraktet definierar vad som skickas, oberoende av implementation.
- API:et validerar mot kontraktet och svarar 400 i stället för att krascha.
- SQLite gör datan persistent – den överlever omstart.
- Vyn konsumerar samma /temperature -endpoint som vilken annan klient som helst.